



1 Types & literals

`int i = 42; long n = 9_000L; _` is a digit separator

`double d = 3.14; and float f = 3.14f;`

`decimal m = 19.99m;` exact — use it for money

`bool ok = true; char c = 'A';` one UTF-16 unit

`int h = 0x2A; int b = 0b101010;` hex, binary

`var names = new List<string>();` type inferred at compile time

`int n = default; O / null / zeroed struct`

`nint p = 42; nuint u = 42;` native-sized integers

`const double Pi = 3.14159;` compile-time, primitives or string

`readonly DateTime _t = DateTime.Now;` runtime constant, set in a ctor

`public required string Name { get; init; }` the object initializer must set it · [C# 11](#)

2 Strings

`($"{greeting} {name}!")` interpolation · [C# 6](#)

`@"C:\Users\Docs"` verbatim: escapes off, newlines kept

`"""{ "a": 1 }"""` raw literal, no escaping · [C# 11](#)

`$$"""{ "n": "{name}" }"""` `$$` makes `{ }` the hole · [C# 11](#)

`ReadOnlySpan<byte> u = "Hi"u8;` UTF-8 literal · [C# 11](#)

`"\e[31mError\e[0m"` `\e` is the ESC escape · [C# 13](#)

`sb.Append($"{Item {i}}, ");` `StringBuilder` beats `+` in a loop

`string.Equals(a, b, StringComparison.Ordinal)` always choose a comparison; `Ordinal` for identifiers

3 Null handling

`int? age; string? name;` nullable value / reference type

`customer?.Name?.Length` whole chain short-circuits to null

`userName ?? "Anonymous"` fallback when null

`userName ??= "Anonymous";` assign only if null · [C# 8](#)

`x is null` · `x is not null` ignores an overloaded `==`

`customer?.Order = GetOrder();` null-conditional assignment; the right side runs only when the left side is non-null · [C# 14](#)

`customer?.Balance += 10;` compound too, but not `++/--` · [C# 14](#)

4 Control flow

`if (t > 30) Hot(); else if (t > 20) Warm();` plus a final `else`

`age >= 18 ? "Adult" : "Minor"` ternary conditional

`switch (day)`
{
 case `DayOfWeek.Saturday`:
 case `DayOfWeek.Sunday`: `Weekend()`; `break`;
 default: `Weekday()`; `break`;
}

every section needs `break` or `return`

`string Kind(object shape) => shape switch`
{
 Circle c when `c.Radius > 10` => "Large circle",
 Rectangle { Width: 0 } => "Line",
 null => "No shape",
 - => "Unknown"
};

`switch` expression with patterns and a `when` guard · [C# 8](#)

5 Loops

`for (int i = 0; i < 10; i++)` known iteration count

`foreach (string n in names)` any `IEnumerable`

`foreach (var (i, n) in names.Index())` index + item · [.NET 9](#)

`foreach (var (key, value) in dict)` deconstruct as you go · [C# 7](#)

`while (cond) { }` may run zero times

`do { } while (cond);` runs at least once

`continue;` · `break;` next iteration / leave loop

`outer: for (int r = 0; r < h; r++)`
 for (int c = 0; c < w; c++)
 if (`grid[r, c].IsGoal`) `break outer`;

labeled break / continue · [C# 15 preview](#)

6 Pattern matching

`value is string text` type pattern + declaration

`person is { Name: "Alice", Age: >= 30 }` property pattern, relational

`order is { Customer: { Name: "Alice" } }` nested property pattern

`x is > 20 and < 30 · s is "a" or "b"` · `x` is not null combinators · [C# 9](#)

`(x, y) switch { (> 0, > 0) => "01", _ => "" }` tuple pattern

`shape is Circle(var radius)` positional — deconstructs a record

`arr is [var first, .., var last]` list pattern with a slice · [C# 11](#)

`nums is [> 0, > 0, ..]` first two positive, rest ignored

`var (name, _) = person;` `var` binds, `_` discards

7 Methods, parameters & lambdas

`public int Add(int a, int b) => a + b;` expression-bodied · [C# 6](#)

`void Greet(string name, string hi = "Hello")` optional parameter with a default

`Greet(hi: "Hi", name: "Alice");` named arguments, any order

`void Swap(ref int a, ref int b)` `ref`: caller sees the change

`bool TryParse(string s, out int result)` `out`: assign before returning

`double Dist(in Point a, in Point b)` `in`: read-only reference · [C# 7.2](#)

`double Len(ref readonly Vector3 v)` like `in`, but the caller should pass a variable · [C# 12](#)

`int Sum(params int[] numbers)` variable-length argument list

`int Sum(params ReadOnlySpan<int> n)` `params` spans and collections, no array alloc · [C# 13](#)

`int Factorial(int n)`
{
 int `Calc(int x) => x <= 1 ? 1 : x * Calc(x - 1);`
 return `Calc(n);`
}

local function: a helper scoped to one method · [C# 7](#)

`Func<int, int, int> add = (a, b) => a + b;` returns a value

`Action<string> log = m => Write(m);` returns void

`(x, y = 10) => x + y` default lambda parameter · [C# 12](#)

`(text, out result) => int.TryParse(text, out result)` lambda parameter modifiers without types · [C# 14](#)

8 Classes, records & structs

`public string Name { get; set; }` auto-implemented property

`public bool IsAdult => Age >= 18;` computed, expression-bodied

`public string Id { get; init; }` set during initialization only · [C# 9](#)

`class Person(string name, int age)` primary constructor — the parameters are not fields · [C# 12](#)

`record Person(string First, int Age);` value equality, `Deconstruct`, `ToString` · [C# 9](#)

`person with { Age = 31 }` copy, changing a few members

`record struct Point(double X, double Y);` value-type record · [C# 10](#)

`readonly record struct Rgb(byte R, byte G);` positional members become `get/init`

`sealed class Cache { }` no further inheritance

`abstract class Shape { }` must be derived from

`static class MathHelpers { }` no instances, no inheritance

9 Interfaces & generics

`public interface ILogger`
{
 void `Log(string message);`
 void `LogError(string m) => Log($"ERROR: {m}");`
}

default interface implementation · [C# 8](#)

`class Repo<T> where T : Employee, new()` generic type with constraints

`where T : struct` // value type
`where T : class` // reference type
`where T : notnull` // never null
`where T : unmanaged` // pointer-friendly
`where T : new()` // ctor — must be last

constraints stack per type parameter

`where T :` allows `ref struct T` may be a `ref struct` like `Span<T>` · [C# 13](#)

`static T Sum<T>(IEnumerable<T> v) where T : INumber<T> => v.Aggregate(T.Zero, (a, b) => a + b);`

generic math over any numeric type · [C# 11](#)

`static abstract TSelf CreateDefault();` static abstract interface member · [C# 11](#)

10 Extension members

`public static string Truncate(this string s, int n) => s.Length <= n ? s : s[..n];`

classic extension method — still valid, same IL

`extension<T>(IEnumerable<T> source)`
{
 public bool `IsEmpty` => !`source.Any()`;
 public `IEnumerable<T> Tail()` => `source.Skip(1)`;
}

extension property and method; the block lives in a top-level `static class` · [C# 14](#)

`extension<T>(IEnumerable<T>)`
{
 public static `IEnumerable<T> Identity` => `[]`;
}

receiver type only, no name: static extension members. Extension operators live here too · [C# 14](#)

`extension(IEnumerable<int> s)`
{
 public int `this[int i]` => `s.ElementAt(i)`;
}

extension indexer · [C# 15 preview](#)

11 Collections & collection expressions

`int[] numbers = [1, 2, 3, 4, 5];` collection expression · [C# 12](#)

`List<string> names = ["Alice", "Bob"];` same for sets, spans, interfaces

`int[] more = [0, .. numbers, 6];` spread splices a collection in

`Dictionary<string,int> d = new() { ["a"] = 1 };` dictionaries have no collection-expression form yet

`d.TryGetValue("b", out int v)` lookup without throwing

`int[] a = [3, 1, 2]; ReadOnlySpan<int> s = a;` first-class spans: implicit, no copy · [C# 14](#)

`numbers[^1] · numbers[1..^1]` `^1` is last · [C# 8](#). Ranges need `Slice`, so `List<T>` takes `^` but has no range slicing

`HashSet<int> set = [1, 2, 3];` plus `UnionWith` / `IntersectWith`

`queue.Enqueue(x); queue.Dequeue();` FIFO

`stack.Push(x); stack.Pop();` LIFO

`List<string> l = [with(capacity: 100), .. names];` collection expression arguments · [C# 15 preview](#)

12 LINQ

`numbers.Where(n => n % 2 == 0)` filter

`people.Select(p => p.Name)` project each element

`people.OrderBy(p => p.City).ThenByDescending(p => p.Age)` sort by several keys

`people.GroupBy(p => p.City)` gives `IGrouping<TKey, T>`

`nums.First(n => n > 5) · nums.Any() · nums.All(IsOk)` element and quantifier operators

`nums.Sum() · nums.Average() · query.ToList()` LINQ is lazy until you materialize

`from n in numbers where n > 5 order by n descending select n * 2`

query syntax, same operators underneath

`nums.Chunk(3) · people.MaxBy(p => p.Age) .NET 6` — also `MinBy`, `DistinctBy`

`words.CountBy(w => w) · people.Index()` · [.NET 9](#) — also `AggregateBy`

`people.LeftJoin(orders, p => p.Id, o => o.CustomerId, (p, o) => (p.Name, o?.Total));`

[.NET 10](#) — `LeftJoin/RightJoin` keep unmatched rows

`numbers.Shuffle()` · [.NET 10](#) — also `Sequence`

13 Async

`async Task<string> GetAsync(string url)` `Task<T>` returns a value, `Task` returns none

`await client.GetStringAsync(url);` frees the thread until it completes

`ValueTask<int> ReadAsync()` when the result is usually ready

`await Task.WhenAll(t1, t2, t3);` concurrent, wait for all

`await Task.WhenAny(t1, t2);` first to finish; await it again

`await foreach (var t in Task.WhenEach(tasks))` handle each result as it lands · [.NET 9](#)

`await Task.Delay(100, cancellationToken);` pass the token all the way down

`cts.CancelAfter(TimeSpan.FromSeconds(5));` then catch `OperationCanceledException`

`await foreach (string x in GetStreamAsync())` consume an `IAsyncEnumerable<T>` · [C# 8](#)

`await SendAsync().ConfigureAwait(false);` library code: don't capture the context

`async void` event handlers only — nothing can catch its exceptions or await it

14 Exceptions

`try { Risky(); }`
`catch (IOException ex) when (IsTransient(ex)) { }`
`catch (Exception ex) { Log(ex); throw; }`
`finally { Cleanup(); }`

the `when` filter runs before unwinding; `finally` always runs · [C# 6](#)

`throw;` keeps the original stack trace

`throw ex;` resets it — rarely what you want

`throw new ArgumentException("Empty.", nameof(data));` always name the parameter

`ArgumentNullException.ThrowIfNull(data);` guard-clause helper · [.NET 6](#)

`public class CustomerNotFoundException(int id)`
 : `Exception($"Customer {id} was not found")`
{
 public int `CustomerId` { get; } = id;
}

custom exception; skip the obsolete serialization ctor

`int.TryParse(s, out int n)` use `Try...` for expected failures

15 Code organization

`namespace MyApp.Services;` file-scoped namespace · [C# 10](#)

`global using System.Linq;` applies to the whole project · [C# 10](#)

`using static System.Math;` then just `Sqrt(x)`

`using Point = (int X, int Y);` alias any type, tuples included · [C# 12](#)

`file class UserValidator { }` visible only in this file · [C# 11](#)

`public` // anywhere
`internal` // this assembly
`protected` // this type and derived types
`private` // this type only
`protected internal` // assembly OR derived
`private protected` // assembly AND derived

access modifiers

`public partial string Conn { get; set; }` partial property · [C# 13](#); partial ctor and event · [C# 14](#)

`#:package Spectre.Console@0.49.1`
`#:property TargetFramework=net10.0`
`AnsiConsole.MarkupLine("[green>Hello[/]");`

one `.cs` file is a whole app: `dotnet run app.cs` · [.NET 10](#)

16 New in C# 14 (.NET 10)

`extension(IEnumerable<int> src) { ... }` extension members: properties, statics, operators

`{ get; set => field = Max(0, value); }` `field` — no backing field to declare

`customer?.Order = GetOrder();` null-conditional assignment

`nameof(List<>)` unbound generics; gives "List"

`ReadOnlySpan<int> s = array;` first-class spans: implicit conversions

`(text, out result) => ...` lambda modifiers without types

`public partial Settings(string path);` partial constructors and events

`public void operator +=(Vector2 v)` instance compound assignment mutates in place

17 C# 15 preview (.NET 11)

`union Pet(Cat, Dog, Bird);` exactly one of the case types

`closed record class Shape;` fixed subclass set, so `switch` is exhaustive without a `_` arm

`break outer;` `continue outer;` labeled jumps

`[with(capacity: 10), .. names]` collection expression arguments

`public int this[int i] => s.ElementAt(i);` extension indexer inside an `extension` block

Preview features need `<LangVersion>preview</LangVersion>` on a .NET 11 preview SDK and can still change.